Data Lifecycle

decide once, as rules; let storage apply them

Watch the 4-min video

In one line: lifecycle rules, filtered by prefix, tag or size, move data to cheaper tiers by age and delete expired objects, old versions and abandoned multipart uploads automatically; retention policies and legal holds override them for data that must be kept, so classify data when it's written and test rules before trusting them.

1 The bill that only grows

Storage grows every month: half-finished uploads, exports from three years ago, logs, test files, and old versions kept by versioning. Nobody deletes anything because nobody is sure what's safe to delete, and a cleanup script written once is forgotten or fails silently. **Data lifecycle management** turns those decisions into declarative rules attached to the bucket, which the storage service applies every day, forever, at no compute cost to you.

2 Lifecycle rules

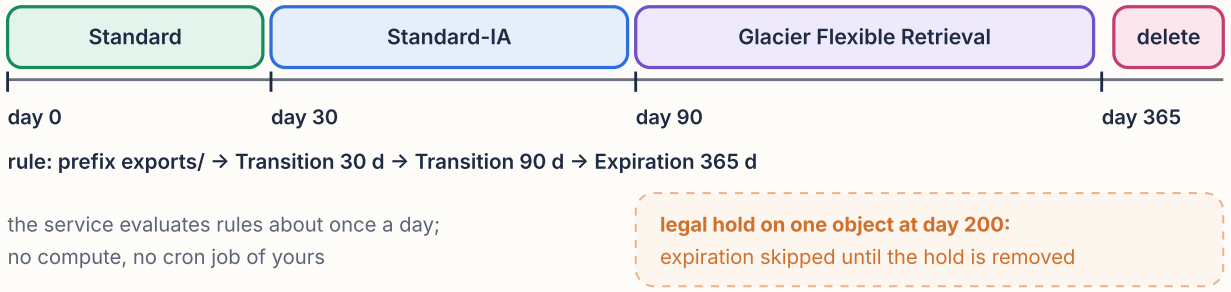


Figure 1. One rule, applied to every object under a prefix, by age. A legal hold on a single object overrides the expiration.

A rule = **filter** + **actions by age**.

- **Filters**: key prefix (`logs/`), object tags (`retention=90d`), object size (S3 `ObjectSizeGreaterThan`), or a combination. Azure filters on blob type, prefix (starting with the container name) and blob index tags.
- **Transition** to a cooler tier after N days (lesson 3). Transitions only go colder; going back needs a copy or rehydration.
- **Expiration / delete** after N days, or on a date.
- **Age**: S3 counts days from creation (rounded to the next midnight UTC); Azure can use modification time, creation time, last access time (with access tracking on) or time since the last tier change.

Execution is asynchronous: S3 evaluates rules about once a day and queues eligible objects, and you stop paying storage for an expired object once it is eligible, even if removal lags. A new or changed Azure policy can take up to 24 hours to take effect. S3 allows up to 1,000 rules per bucket. Constraints worth knowing: S3 won't transition to Standard-IA before an object is 30 days old, by default doesn't transition objects under 128 KB, and charges per transition request, which can make tiering millions of tiny objects a loss.

3 Old versions and incomplete uploads

Two rules alone often remove a surprising share of a bill:

- **Noncurrent versions**: with versioning on, every overwrite and delete keeps the previous version (lesson 3). A file rewritten daily keeps 365 versions a year. `NoncurrentVersionExpiration` (with `NoncurrentDays`, optionally keeping the newest N via `NewerNoncurrentVersions`) deletes them after a window; Azure has equivalent actions on `version` and `snapshot`.
- **Incomplete multipart uploads**: parts of abandoned uploads are invisible in listings but billed (lesson 6). `AbortIncompleteMultipartUpload` after a few days removes them. (Azure discards uncommitted blocks automatically after about a week.)
- **Expired delete markers**: after old versions are gone, a delete marker with nothing behind it is clutter that slows listings; `ExpiredObjectDeleteMarker` removes it.

4 Retention policies

Deletion is half the job; the other half is **not** deleting what must be kept. Financial records are typically kept for several years (the exact period depends on the law and country), audit logs as long as regulations require. A **retention policy** makes data undeletable and unmodifiable until a date, even by administrators, so a mistake, a rogue script or an attacker can't erase records early:

	Retention	Legal hold
Amazon S3 Object Lock	per version or bucket default; governance or compliance mode	per-version on/off flag, no expiry
Azure immutable storage	time-based policy on container or version; locked policies can only be extended	tag-based hold on container or version
Google Cloud Storage	bucket retention policy, lockable (Bucket Lock)	object holds: *temporary* and *event-based*

Retention that starts at an **event** rather than at creation ("keep 7 years after the account closes") needs either an event-based hold (GCS) or your application setting a retain-until date on the objects when the event happens.

5 Legal hold

When data is relevant to a lawsuit, audit or investigation, a **legal hold** freezes it with no end date, overriding every lifecycle rule and retention expiry until it is explicitly removed, typically by the legal team through a separately permissioned role (S3: `s3:PutObjectLegalHold`). Your rules don't need special cases: the platform skips held objects.

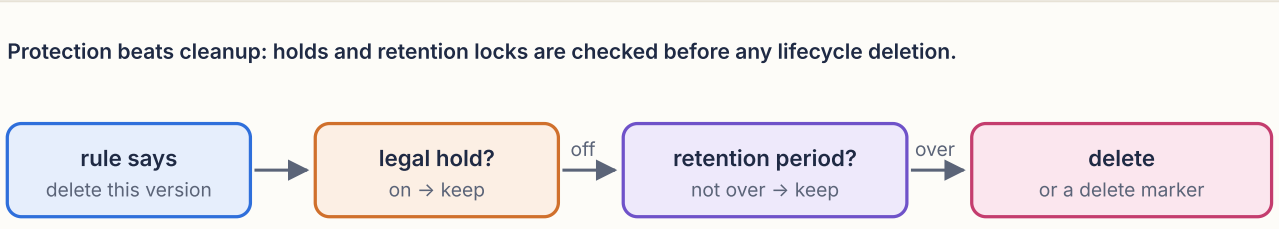


Figure 2. Order of precedence when a lifecycle rule wants to delete a version.

Watch the conflict with privacy law: a GDPR erasure request can't remove data under a legal retention obligation (Article 17(3)(b) allows keeping it), but you must then restrict its use and delete it when the obligation ends. Lifecycle rules are how that "when it ends" actually happens.

6 Know your data first

- **Classify at write time**: put data type and retention class in the key prefix (`temp/`, `exports/`, `records/`) or in tags (S3 allows 10 tags per object; Azure blob index tags are also queryable). Rules can only act on what they can match; classifying billions of existing objects later is expensive.
- **Measure**: S3 Storage Lens and Inventory, Storage Class Analysis, Azure blob inventory: find the biggest, oldest prefixes, noncurrent bytes and incomplete uploads.
- **Test safely**: a wrong delete rule is a very efficient way to lose data. Apply it to a test prefix first, use an inventory report to count what a rule would match, roll out transitions before deletions, keep versioning on with a noncurrent window as a safety net, and review rules like code (infrastructure as code, peer review).

7 Example: delete temp uploads after 7 days

```
{ "Rules": [
  { "ID": "expire-temp",  "Status": "Enabled", "Filter": { "Prefix": "temp/" },
    "Expiration": { "Days": 7 } },
  { "ID": "abort-mpu",    "Status": "Enabled", "Filter": {},
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 3 } },
  { "ID": "old-versions", "Status": "Enabled", "Filter": {},
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "Expiration": { "ExpiredObjectDeleteMarker": true } } ] }
```

Figure 3. S3 lifecycle configuration for the example. `Filter: {}` applies a rule to the whole bucket.

WORKED EXAMPLE: TEMP UPLOADS, ABANDONED UPLOADS, OLD VERSIONS

Uploads land under `temp/`; when processing succeeds the app copies the file to `media/` and deletes the temp copy (lessons 4, 9). Anything still in `temp/` after 7 days is an abandoned or failed upload. Rule 1 expires `temp/` objects after 7 days. Rule 2 aborts multipart uploads not completed within 3 days. Because the bucket is versioned, rule 1 only adds delete markers, so rule 3 removes noncurrent versions after 30 days (a month to recover from mistakes) and cleans up the leftover markers. If 2,000 uploads a day are abandoned at 50 MB each, that's 100 GB a day, ~36 TB a year that would otherwise pile up; with the rules, temp data stays bounded at about a week of abandoned uploads (plus the 30-day version window). No cleanup job to maintain.

TRADE-OFFS

- Aggressive expiry saves money but risks deleting data someone needed; retention windows and versioning buy recovery time at a cost.
- Prefix-based rules are simple; tag-based rules are flexible but need tags written reliably at upload.
- Locked retention gives strong compliance guarantees and removes your own ability to fix mistakes.
- Daily asynchronous execution is free and hands-off, but nothing happens at an exact time.

COMMON MISTAKES

- Versioning on, no noncurrent-version rule: storage silently doubles.
- No rule for incomplete multipart uploads.
- Expecting an expiration rule to free space in a versioned bucket (it only adds delete markers).
- Shipping a broad delete rule without testing its filter on real inventory.

8 Key terms

Lifecycle rule: filter + age-based transition or deletion actions

Delete marker: placeholder that hides an object in a versioned bucket

Legal hold: open-ended freeze that overrides everything until removed

Noncurrent version: an older version kept by versioning

Retention policy: no delete or overwrite until a date (WORM)

Storage inventory: scheduled report listing objects and their properties

9 Interview questions

Q1. How do you automate storage cost control? BASIC

Lifecycle rules filtered by prefix or tag: move data to cheaper tiers by age, and delete expired data, old versions and incomplete multipart uploads.

Q2. What can override lifecycle rules? BASIC

Retention policies and legal holds, which protect data that must be kept, whatever its age; the platform skips locked or held versions.

Q3. Storage doubled after enabling versioning. Why, and what do you add? SCENARIO

Every overwrite and delete keeps old versions; add a NoncurrentVersionExpiration rule (e.g. 30 days, keep the newest few) and clean up expired delete markers.

Q4. Records must be kept 7 years after a customer closes their account. How? SCENARIO

Retention starting at an event: an event-based hold (GCS) or the app setting a retain-until date when the account closes, plus a rule that deletes once retention ends.

Q5. How would you introduce a delete rule on a 2 PB bucket safely? SCENARIO

Use inventory to measure exactly what the filter matches, apply it to a test prefix, start with transitions, keep versioning with a noncurrent window, and review the rule as code.

Q6. An expiration rule ran but the bucket size didn't drop. Why? TRICKY

The bucket is versioned: expiration added delete markers and turned current versions into noncurrent ones; only a noncurrent-version rule removes the bytes.

Q7. A user exercises the GDPR right to erasure, but the data is under a 7-year legal retention. What happens? TRICKY

The legal obligation takes precedence (GDPR Art. 17(3)(b)); restrict processing now and delete when retention ends, which the lifecycle rules enforce.